Rapid oral drill: 15 questions

---

1. Why must equals and hashCode agree or why must you override both equals and hashCode?

The contract between `equals()` and `hashCode()`

If two objects are equal according to `equals()`, they must have the same `hashCode()`.

The reverse is not required: Two objects can have the same hash code but still be unequal. This is called a **hash collision**.

```
a.equals(b) == true
    => a.hashCode() == b.hashCode()
```

Hash-based collections like HashMap and HashSet rely on both methods to find and identify objects correctly.

While `equals()` defines logical equality, `hashCode()` helps hash-based collections locate objects efficiently.

Their contract ensures that logically equal objects can be found in the same bucket and correctly identified by `equals()`.

What happens if you violate the contract?

Suppose you create a Person class that overrides equals () but not hashCode ().

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Person)) return false;
        Person other = (Person) obj;
        return name.equals(other.name);
    }
}
```

Now consider:

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.
```

```
println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // Usually false
```

Why? Because the inherited `Object.hashCode()` generally uses object identity rather than the name field. The two objects are logically equal but may have different hash codes.

The consequence in a `HashSet`:

```
Set<Person> people = new HashSet<>();

people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // Can be 2, not 1
```

You expected one person because both objects represent Alice. But the set may put them in different hash buckets and fail to recognize that they are equal.

Similarly, a `HashMap` may fail to retrieve a value using a different but logically equal key.

The correct implementation:

Override both methods using the same fields that define logical equality.

```
import java.util.Objects;

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person)) return false;
```

```
        Person other = (Person) obj;
        return Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}
```

Now:

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.

println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // true

Set<Person> people = new HashSet<>();
people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // 1
```

How HashMap uses both methods?

```
1. Calculate hashCode()
Determine the hash bucket to search.
    |
    v
2. Find candidate entries
Use the hash to narrow down the search.
    |
    v
3. Compare with equals()
```

`Determine whether a candidate key is actually equal.`

If equal objects produce different hash codes, the map may search the wrong bucket and never compare those objects using equals ().

---

2. Why is volatile count++ unsafe?

Interview response: volatile guarantees visibility, not atomicity.

Since count++ is a `read-modify-write` operation, multiple threads can overwrite each other's updates.

Use AtomicInteger or synchronization when you need a correct shared counter.

Explanation:

Consider this Java code:

```
private volatile int count = 0;

// Called by multiple threads
count++;
```

Although count++ looks like one operation, it actually involves three steps:

1. Read the current value of count.
2. Increment the value by one.
3. Write the new value back to count.

These steps are not atomic.

In Java, volatile provides two important guarantees:

- **Visibility:** When one thread writes a new value to a volatile variable, other threads can observe that write.
- **Ordering:** It establishes memory-ordering guarantees around volatile reads and writes.

However, it does not make compound operations such as count++ atomic.

Think of it this way:

- `volatile` makes individual reads and writes visible.
- `synchronized` or `atomic` classes can make the entire increment operation safe.

How to fix it?

Option A: Use AtomicInteger (usually the simplest choice)

```
import java.util.concurrent.atomic.AtomicInteger;
```

```java
private final AtomicInteger count = new AtomicInteger(0);

public void increment() {
    count.incrementAndGet();
}
```

`incrementAndGet()` performs the increment atomically, so concurrent updates are not lost.

Option B: Use synchronized

```java
private int count = 0;

public synchronized void increment() {
    count++;
}
```

Only one thread at a time can execute this synchronized method on the same object, preventing concurrent increments from interfering.

Option C: Use volatile alone

```java
private volatile boolean shutdown = false;
```

This is appropriate when you need visibility for simple reads and writes, such as a shutdown flag:

---

3. When does ConcurrentHashMap still need higher-level coordination?

Interview Response:

ConcurrentHashMap protects its internal data structure and provides atomic operations at the mapping level.

Higher-level coordination is needed when correctness depends on :

1. multiple operations
2. multiple keys,
3. external state, or
4. consistent global view.

Common situations requiring higher-level coordination

1. **Check-then-act operations**

   Even though every map operation is thread-safe, the entire check-and-update sequence is not atomic.

```
if (!map.containsKey(key)) {
  map.put(key, createValue());
}
```

Another thread can insert the key between the check and the insertion.

Use `putIfAbsent()` or `computeIfAbsent()` when appropriate:

```
map.computeIfAbsent(key, k -> createValue());
```

Note: The mapping function should be short and should not attempt to update other mappings in the same map.

2. **Multiple keys must change together**

   Suppose a bank transfer moves money between two accounts. Each update may be atomic for its own key, but the transfer as a whole isn't atomic. You may need a shared lock, a transaction mechanism, or another design that coordinates both accounts.

3. **A global invariant must remain true**

   Imagine a system where the sum of all allocations must never exceed 100: Making map a `ConcurrentHashMap` does not coordinate the map update with `totalAllocated` or make the capacity check atomic.

```
if (totalAllocated < 100) {
    map.put(id, allocation);
    totalAllocated += allocation;
}
```

4. **A multi-step lifecycle must happen only once**

   For example, multiple threads might attempt to initialize a service, register a resource, and publish its status. Atomicity of one map entry doesn't automatically coordinate all those steps.

   Use an appropriate lifecycle mechanism, such as a Future, a lock, or an explicit state machine.

---

4. What is the difference between thenApply and thenCompose?

In Java's CompletableFuture, the main difference is:

- `thenApply()`transforms the result of a future into another value.

```
CompletableFuture<String> future =
        CompletableFuture.supplyAsync(() -> "Hello")
                .thenApply(s -> s + " World");

System.out.
```

```
println(future.join()); // Hello World
```

- `thenCompose()` transforms the result into another future and flattens it into a single future.

---

5. Why can JDBC block a Vert.x event loop?

How the Vert.x event loop works

A Vert.x event loop typically handles events for many connections on a small number of threads. As such, they depend on handlers returning quickly so it can continue processing other requests, timers, and I/O events.

Traditional JDBC operations are synchronous (blocking). So, even if the database is healthy, network latency, connection-pool waits, locks, or a slow query can keep the thread blocked.

The problem isn't necessarily the database being slow. It's that the event loop is waiting synchronously.

How to fix it?

Option A: Use executeBlocking ()

```
vertx.executeBlocking(() -> {
    return jdbcQuery(); // Runs on a worker thread
}).onSuccess(result -> {
    // Handle result on the original Vert.x context
    request.response().end(result);
}).onFailure(err -> {
    request.response().setStatusCode(500).end();
});
```

This keeps the JDBC call off the event loop. Vert.x runs the blocking callable in its worker pool and completes the returned future on the original context.

Option B: Use a reactive database client

For a more naturally non-blocking architecture, use an appropriate Vert.x reactive database client, such as a reactive SQL client supported for your database.

```
pool.preparedQuery("SELECT name FROM users WHERE id = ?")
    .execute(Tuple.of(userId))
    .onSuccess(rows -> {
        // Handle results without blocking the event loop
    });
```

Here, the database operation is asynchronous: the event loop can process other events while waiting for the database response.

Does wrapping JDBC in a CompletableFuture solve it? Not by itself.

This is still dangerous when called from a Vert.x event loop:

```
CompletableFuture.supplyAsync(() -> jdbcQuery())
   .join();
```

Although `supplyAsync()` usually runs the JDBC operation on a different thread, `join()` blocks the calling thread while waiting for the result. If that calling thread is an event loop, you still have the same problem.

Instead, compose asynchronous stages without blocking: This avoids blocking the event loop, provided the callback itself doesn't perform blocking work.

```
vertx.executeBlocking(() -> jdbcQuery())
.onSuccess(result -> {
// Continue asynchronously
});
```

Why does this matter for performance?

Latency: Other requests on the blocked event loop wait longer.

Throughput: Fewer requests can be processed per second.

Scalability: If all event loops become blocked, the entire Vert.x instance can become unresponsive.

Diagnostics: Vert.x can log warnings such as vertx-eventloop-thread-3 has been blocked

---

6. What does Spring Boot add to Spring?

Spring Boot makes Spring applications easier to configure, run, and deploy.

Spring provides the core framework and features; Spring Boot adds sensible defaults, automatic configuration, and tools that reduce the amount of manual setup.

There are five major additions.

1. **Auto-configuration** Spring Boot automatically configures beans and components based on the libraries on your classpath and the settings you provide.
2. **Starter dependencies** Starters bundle commonly used dependencies so you don't have to select every library individually.

   For example, `spring-boot-starter-web` brings together the dependencies commonly needed for a Spring MVC web application.

3. **Embedded servers** Boot can package an application with an embedded server, such as Tomcat, so you can run it directly with java -jar instead of deploying a WAR to a separately installed server.

4. **Production-ready features** Spring Boot Actuator provides features such as health checks, application metrics, and diagnostic endpoints, depending on configuration.

5. **Externalized configuration and conventions** Boot makes it convenient to configure applications through `application.properties`, `application.yaml`, environment variables, and command-line arguments. It also provides sensible defaults that you can override.

---

7. Why might @Transactional not take effect?

Spring's transaction management is usually proxy-based. The method must be invoked through the `Spring-managed proxy` for the transaction advice to run.

Here are the most common causes, how to identify them, and how to fix them.

1. **Self-invocation (the most common cause)** Calling a transactional method from another method in the same class directly using `this` and bypasses the Spring proxy.

```java
@Service
public class OrderService {

    public void processOrder() {
        saveOrder(); // Self-invocation: transaction may not start
    }

    @Transactional
    public void saveOrder() {
        // Database operations
    }
}
```

Fix: Move transactional method into another Spring-managed service and inject it, or call the method through a proxy.

```java
@Service
public class OrderWriter {

    @Transactional
    public void saveOrder() {
        // Database operations
    }
}
```

```java
@Service
public class OrderService {

    private final OrderWriter orderWriter;

    public OrderService(OrderWriter orderWriter) {
        this.orderWriter = orderWriter;
    }

    public void processOrder() {
        orderWriter.saveOrder(); // Goes through the proxy
    }
}
```

2. **The method is not eligible for proxy interception**

   Depending on the proxy mechanism and Spring version, method visibility and class structure can prevent interception.

   - **Private methods:** Cannot be intercepted by standard proxy-based transaction management.
   - **Final methods:** Cannot be overridden by class-based proxies, so transaction advice cannot be applied to them.
   - **Final classes:** Cannot be subclassed by class-based proxies, although interface-based proxies may still work.
   - **Static methods:** Are not intercepted by standard Spring transaction proxies.

   Fix: Use an eligible, typically public, instance method on a non-final Spring-managed class. Package-private and protected methods can work with class-based proxies in modern Spring versions, but behavior depends on the proxy strategy and configuration.

3. **The object was created with new** Spring can apply transactions only when the invocation passes through the relevant Spring proxy.

   ```java
   OrderService service = new OrderService(); // Not Spring-managed
   service.saveOrder();
   ```

   Fix: Let Spring create and inject the bean. Inject it into the calling component instead of constructing it manually.

   ```java
   @Service
   public class OrderService {

       @Transactional
       public void saveOrder() {
   ```

```
            // ...
        }
    }
```

4. **The exception does not trigger a rollback**

   By default, Spring rolls back on `RuntimeException` and `Error`, but not on **checked exceptions**.

   ```
   @Transactional
   public void saveOrder() throws IOException {
        repository.save(order);
        throw new IOException("Failed");
   }
   ```

   The transaction may commit despite the checked exception, depending on how the exception is handled and transaction completion proceeds.

   Fix: Specify rollback rules when needed.

   ```
   @Transactional(rollbackFor = Exception.class)
   public void saveOrder() throws IOException {
       repository.save(order);
       throw new IOException("Failed");
   }
   ```

   Also watch for exceptions that are caught and swallowed:

   ```
   @Transactional
   public void saveOrder() {
        try {
            repository.save(order);
            performOperation();
        } catch (Exception e) {
            log.error("Operation failed", e);
            // No exception escapes; Spring may commit
        }
   }
   ```

   If the transaction should roll back, rethrow the exception or explicitly mark the transaction rollback-only.

5. **Transaction management is not configured correctly**

6. **The wrong transaction manager is being used**

Applications with multiple data sources or transaction managers can accidentally start a transaction against the wrong resource.

```
@Transactional("ordersTransactionManager")
public void saveOrder() {
```

```
    // ...
}
```

Fix: Specify the correct transaction manager explicitly, if necessary, and verify that the repository and transaction manager operate on the intended data source.

Also, transactions do not automatically span arbitrary independent databases or external services. That requires appropriate distributed transaction infrastructure or a different consistency strategy.

7. **Transaction propagation changes the behavior**

Propagation settings can cause a method to join an existing transaction or start a separate one.

- `REQUIRED` — joins an existing transaction or creates one.
- `REQUIRES_NEW` — suspends an existing transaction and starts a new one.
- `SUPPORTS` — participates in a transaction if one exists, but does not require one.
- `NOT_SUPPORTED` — executes without a transaction, suspending an existing one if applicable.
- `NEVER` — requires that no transaction exists.

For example:

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void updateOrder() {
    // Executes without a transaction
}
```

Fix: Check the propagation setting on both the caller and the called method, not just the annotation on the method you're debugging.

8. **The transaction is active, but the data is not rolled back**

Sometimes `@Transactional` works correctly, but the observed behavior makes it appear ineffective.

Common reasons include:

- The operation uses a different database connection or data source.
- A database operation is performed through a resource not enlisted in the transaction.
- The database or storage engine does not support the expected transactional behavior.
- The code commits manually or uses autocommit outside Spring's transaction management.

- The code reads from a replica or another connection and observes different state.
- The transaction commits before an asynchronous operation executes.

Fix: Verify the actual connection, transaction manager, database resource, and transaction boundaries. Do not assume that annotating a method makes every operation it invokes transactional.

9. **Async execution or thread boundaries**

Imperative Spring transactions are generally thread-bound. They do not automatically propagate to another thread.

```
@Transactional
public void processOrder() {
    asyncService.processAsync(); // Runs on another thread
}
```

If `processAsync()` uses @Async, its work generally runs outside the caller's transaction unless it establishes its own transaction.

Fix: Put `@Transactional` on the method executed by the async worker if that work needs a transaction. For reactive applications, use the appropriate reactive transaction manager and preserve the reactive execution context.

10. **The wrong annotation or framework is being used**

Make sure the annotation matches the transaction infrastructure:

- `org.springframework.transaction.annotation.Transactional` — Spring's annotation.
- `jakarta.transaction.Transactional` — Jakarta Transactions annotation, supported in appropriate Spring configurations.
- `javax.transaction.Transactional` — legacy Java EE annotation, relevant to older stacks.

Also verify whether you are using Spring JDBC/JPA, JTA, or reactive persistence, since their transaction requirements differ.

---

8. How do you detect and fix N+1 queries?

---

9. How do you prevent lost updates?

---

10. Why can a LEFT JOIN unexpectedly remove unmatched rows?

A LEFT JOIN can unexpectedly remove unmatched rows when a condition in the WHERE clause filters out the NULL values generated for rows that have no match on the right-hand table.

The most common cause is putting a condition on the right-hand table in WHERE instead of ON.

---

11. How do you invalidate Redis data safely enough for the use case?

The safest way to invalidate Redis data depends on

- **how stale the data can be**,
- **how expensive it is to rebuild**, and
- **what happens if a stale value is served briefly**.

The general principle is:

- invalidate narrowly,
- make stale data detectable, and
- ensure concurrent requests cannot repopulate the cache with outdated values.

---

12. What must you validate in a JWT besides its signature?

---

13. How do you prevent cross-tenant access?

Use defense-in-depth approach.

The core rule is:

Every request must be authenticated, authorized, and enforce tenant isolation at every data-access boundary.

Never rely on a single tenant_id filter or frontend access control.

1. **Key security controls**

Layer	Control	Purpose
Authentication	Validate identity and session	Identify the caller
Authorization	Verify tenant membership and permissions	Prevent unauthorized tenant access
API	Enforce tenant scope server-side	Prevent IDOR/BOLA attacks

Layer	Control	Purpose
Database	Row-Level Security (RLS) or tenant-isolated databases	Block cross-tenant queries
Cache	Namespace keys by tenant	Prevent cached data leakage
Storage	Tenant-scoped object access	Protect files across tenants
Background Jobs	Carry and validate tenant context	Prevent asynchronous data leakage
Monitoring	Log and alert on tenant access violations	Detect attempted or accidental breaches

2. **Enforce tenant isolation on every request** A secure request flow looks like this:

```
Client Request
      |
      v
Authenticate User
      |
      v
Resolve Tenant Context
      |
      v
Verify Tenant Membership
      |
      v
Check Resource Permissions
      |
      v
Apply Database Tenant Isolation
      |
      v
Return Authorized Data
```

Important: Never trust a tenant_id supplied by the client without verifying that the authenticated user is allowed to access that tenant.

3. **Example: Safe database access**

Unsafe — resource ID only

```
SELECT *
FROM documents
WHERE id = :document_id;
```

Safer — tenant-scoped query

```
SELECT *
```

```
FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

The verified_tenant_id must come from a trusted, authorized server-side context.

For PostgreSQL, add `Row-Level Security (RLS)` as another protection layer:

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE
POLICY tenant_isolation
ON documents
USING (
  tenant_id = current_setting('app.tenant_id')::uuid
)
WITH CHECK (
  tenant_id = current_setting('app.tenant_id')::uuid
);
```

Set the tenant context from the verified identity within the database transaction. Use an application database role that cannot bypass RLS, and reset transaction context before reusing connections.

PostgreSQL documents important RLS exceptions, including superusers and roles with BYPASSRLS.

4. **Test cross-tenant isolation** At minimum, test these scenarios:

   - Tenant A cannot read Tenant B's records.
   - Tenant A cannot update or delete Tenant B's records.
   - Changing object IDs or tenant IDs does not bypass authorization.
   - Cache and file access cannot expose another tenant's data.
   - Reused database connections do not retain a previous tenant's context.
   - Background jobs cannot run under an incorrect tenant context.

5. **Recommended approach** For a SaaS application with a shared database, use three independent controls:

   1. **Application:** Centralized, deny-by-default authorization.
   2. **Database:** Tenant-scoped queries backed by RLS.
   3. **Testing:** Automated cross-tenant negative tests in CI/CD.

   The main principle is **tenant isolation must remain effective even when one application-layer check is missed.**

How tenant scope is set Tenant scope is set by the **backend after authenticating the user and verifying their access to the tenant.**

The frontend may request a tenant, but it must never decide which tenant the user is authorized to access.

1. **Request flow**

```
Client Request
    |
    | Authorization: Bearer <JWT>
    | X-Tenant-ID: tenant_123
    v
API Gateway / Backend
    |
    v
1. Validate JWT
    |
    v
2. Identify user
    |
    v
3. Verify user belongs to tenant_123
    |
    v
4. Create trusted TenantContext
    |
    | user_id   = user_456
    | tenant_id = tenant_123
    | role      = admin
    v
5. Enforce tenant scope
    |
    +---- API authorization
    |
    +---- Database queries / RLS
    |
    +---- Cache and storage
    |
    v
Return tenant-specific data
```

2. **Where does tenant_id come from?**

   Select tenant from token, session, URL, or header. Store verified tenant_id server-side

   Important: A valid JWT proves the token is authentic. It does not automatically prove access to every tenant requested by the caller.

3. **How is tenant scope applied to the database?** For `PostgreSQL with Row-Level Security (RLS)`, tenant scope is typically set **per transaction**.

```
BEGIN;
-- Set verified tenant context
SELECT set_config(
'app.tenant_id',
:verified_tenant_id,
true
);

-- RLS automatically filters rows
SELECT * FROM documents;

COMMIT;
```

The `true` argument makes the setting transaction-local, preventing tenant context from unintentionally carrying into a later transaction on a reused connection.

This assumes RLS policies are configured and the database role cannot bypass them.

---

14. What does a payment timeout actually tell you?

---

15. What did you personally implement in your strongest project?
